

Day 5 — Wed 9 Sep

Countdown: 17 days to General English & GK

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

General — amplification

Source: General/01_GENERAL_PAPER_PLAYBOOK.md

4. AMPLIFICATION — 10 marks

4.1 What amplification is (most candidates get this wrong)

You are given a **proverb, maxim, quotation or short saying** — for example "A stitch in time saves nine" or "Knowledge is power" — and asked to **amplify** it.

Amplification means: expand this compressed statement into a developed piece of prose that explains what it means, why it is true, and where it applies.

It is **not**:

- ✗ a mini-essay on a topic (you must stay tethered to the saying);
- ✗ a definition (too short);
- ✗ a story (an example is one *part* of it, not the whole);
- ✗ a précis (that is compression; this is expansion — the opposite).

Think of it as: *the saying is a seed; you grow it into a plant of five parts.*

4.2 Length and format

Words	200–300 (250 is the sweet spot)
Paragraphs	3–5 short paragraphs, or one well-signposted block
Time	20 minutes total
Opening	Quote the maxim itself in the first line, in quotation marks
Heading	Write the maxim as the heading, underlined

Do not write 600 words. Amplification is worth 10 marks — the same as grammar. Match your effort to the marks.

4.3 The five-move structure (memorise this)

Move	What you write	~Words
1. Literal meaning	What the words say on their face. Unpack the image or metaphor concretely.	40
2. Deeper / general meaning	The universal principle behind the image. This is the heart of the answer.	60
3. Illustration	One or two concrete examples — historical, everyday, scientific, or from public life. This is where most candidates are weakest and where marks are easiest.	70
4. Contemporary relevance	Why this matters now — link to modern life, technology, governance, society, India today.	50

The single most useful tip: move 3 (illustration) is what separates a 5/10 from an 8/10. Have a stock of examples ready that can be bent to almost any maxim — see §4.5.

4.4 Worked example 1 — "A stitch in time saves nine"

"A stitch in time saves nine"

Taken literally, the proverb describes a piece of cloth in which a single thread has given way. If the small tear is mended at once, one stitch is enough. If it is ignored, the tear widens with every use until nine stitches, or a new garment altogether, are required. The image is domestic, but the logic is universal.

The deeper meaning is that problems have a cost curve that rises with delay. Difficulty is rarely static; neglect compounds it. A small defect attended to early consumes a little effort at a moment when effort is cheap. The same defect left alone draws in other systems, hardens into habit, and eventually demands resources far beyond what timely action would have required. Promptness, the proverb suggests, is not merely a virtue of character but a form of economy.

Illustrations are everywhere. In medicine, a condition detected at screening is often curable, while the same condition detected late may be untreatable at any price — which is precisely the argument for preventive health programmes. In engineering, a hairline crack in a bridge or a small unpatched flaw in software costs little to repair and a great deal to ignore; most large system failures begin as a small fault that nobody thought worth attending to. In public administration, a minor local grievance addressed within a week may never be heard of again, while the same grievance ignored for a year can become a movement.

The saying has gained rather than lost force in our own time. Modern systems are tightly coupled, so a small failure propagates faster than it once did. Climate change is perhaps the largest example of a stitch not taken in time: emissions that could have been curbed cheaply decades ago now demand a far costlier global effort. The same holds in personal life, where a postponed examination syllabus or an unattended savings habit exacts its price later.

In short, the proverb counsels vigilance rather than mere haste. To act early is not to act anxiously; it is to recognise that time itself is an ingredient in the cost of every problem. The stitch taken today is the cheapest stitch we will ever take.

(≈340 words — trim the third paragraph to two examples for a 250-word version.)

4.5 Worked example 2 — "Knowledge is power"

"Knowledge is power"

The phrase, associated with Francis Bacon, sets two words side by side that at first seem unrelated. Literally it claims that knowing something — how a thing works, what the law says, where the money went — confers the capacity to act upon it. Power here is not force but the ability to bring about intended effects.

At a deeper level the maxim describes the relationship between information and control. The person who understands a system can predict it; the person who can predict it can shape it. This is why, historically, knowledge has been guarded: priesthoods restricted scripture, guilds restricted craft secrets, and colonial administrations restricted technical education. Conversely, every genuine expansion of freedom in history has been accompanied by an expansion of literacy. Ignorance is

not a neutral absence; it is a condition in which one is acted upon rather than acting.

The illustrations are striking. The scientific revolution converted knowledge of nature into power over nature — medicine, electricity, agriculture. In India, the Right to Information Act, 2005 rests on exactly this proposition: it gives the ordinary citizen access to official records, and in doing so shifts a measure of power from the office to the applicant. At an individual level, a farmer who knows the market price of his crop bargains differently from one who does not.

In the present century the maxim needs a qualification. We live amid an unprecedented abundance of information, yet abundance is not knowledge. Data must be verified, understood and applied before it becomes power; unverified information can instead be used to manipulate, as the spread of misinformation online demonstrates. Moreover, knowledge concentrated in a few hands — in large technology firms or in the state — produces power that is equally concentrated. The democratic task of our time is therefore not only to create knowledge but to distribute it.

Knowledge, then, is power only when it is true, understood, and shared. Held by the few it is dominance; held by the many it is freedom.

(≈330 words)

4.6 Worked example 3 — *"United we stand, divided we fall"*

"United we stand, divided we fall"

The literal picture is military. A body of people standing together presents an unbroken front and cannot easily be pushed over; the same people scattered are overcome one by one. The saying compresses a fact about physical formations into a rule about human affairs.

Its deeper claim is that collective strength is not simply the sum of individual strengths. Unity adds something — coordination, mutual support, shared purpose — that the same individuals acting separately do not possess. Division subtracts correspondingly, because a divided group spends against itself the energy it would otherwise spend on its common problem. This is why those who wish to weaken a community rarely attack it directly; they set its parts against one another.

History supplies both halves of the lesson. India's freedom struggle became irresistible only when it drew together regions, languages, faiths and classes into a single movement; the same subcontinent had earlier been subdued piecemeal because its kingdoms would not combine. The formation of cooperative societies, of trade unions, and of self-help groups follows the same principle at a smaller scale. In the North-East, the practical value of unity is visible in the village institutions that manage common forests, water sources and footpaths — assets no single household could maintain but which a community sustains without difficulty.

Today the maxim applies to problems that respect no boundary. Pandemics, climate change and cybercrime cannot be answered by any one state acting alone, which is why international cooperation, however imperfect, persists. Within a country, unity means something more demanding than uniformity: it means holding together across real differences of tribe, language and belief, and resisting the easy politics that profits from sharpening those differences.

Unity, properly understood, is therefore not the absence of difference but the presence of a shared purpose strong enough to carry difference. That is what allows a people to stand; its loss is what makes them fall.

(≈325 words)

4.7 A stock of illustrations you can bend to almost any maxim

Have these ready. In the hall you will be able to attach at least two to whatever maxim appears:

Domain	Reusable illustration
Science / medicine	Preventive screening vs late diagnosis; vaccination; the scientific method as patient accumulation
Technology	A small software bug that brings down a large system; the digital divide; misinformation online
Indian history	The freedom struggle as unity across differences; Gandhi's insistence on means as well as ends; social reform movements
Governance	The Right to Information Act, 2005; MGNREGA as a right rather than a favour; decentralisation through Panchayati Raj
Nagaland / North-East	Village councils and community management of common resources; the Communitisation Act, 2002; jhum cultivation and the cost of short-term choices; Hornbill Festival and cultural unity in diversity
Environment	Climate change as delayed action; deforestation; the tragedy of the commons
Everyday life	Studying steadily vs cramming; savings and compound interest; maintenance of a house or a vehicle

4.8 Fifteen maxims most likely to appear

Prepare a 3-line mental plan for each — literal, deeper, one example.

1. A stitch in time saves nine
2. Knowledge is power
3. United we stand, divided we fall
4. Honesty is the best policy
5. Where there is a will, there is a way
6. Necessity is the mother of invention
7. Health is wealth
8. Time and tide wait for no man
9. Look before you leap
10. Actions speak louder than words
11. A friend in need is a friend indeed
12. All that glitters is not gold
13. Failure is the pillar of success
14. Simple living, high thinking
15. Service to man is service to God

4.9 Amplification checklist

- [] Maxim written as the heading, underlined
- [] Maxim quoted in the first sentence
- [] Literal meaning stated before the deeper meaning
- [] At least **two** concrete illustrations
- [] Explicit contemporary/Indian relevance
- [] Conclusion restates the maxim in fresh words
- [] 200–300 words, not 600
- [] Did not drift into a general essay on the topic

● Core — combinational circuits

Source: `_Shared/CORE_01_DigitalLogic.md`

D0. Combinational vs sequential — the framing distinction

	Combinational	Sequential
Output depends on	Present inputs only	Present inputs and past history (state)
Memory	None	Has memory elements (flip-flops)
Clock	Not required	Usually clocked (synchronous)
Feedback	No feedback path	Feedback present
Described by	Truth table / Boolean expression	State table / state diagram / excitation table
Examples	Adder, MUX, decoder, encoder, comparator, code converter	Flip-flop, register, counter, shift register, RAM

Design procedure for combinational circuits (the 5 steps — quote these verbatim):

1. State the problem and determine the number of inputs and outputs.
2. Assign letter symbols to inputs and outputs.
3. Derive the **truth table**.
4. Obtain the **simplified Boolean expression** for each output (K-map / algebra).
5. Draw the **logic diagram** and verify it.

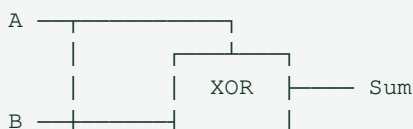
D1. Half adder and full adder

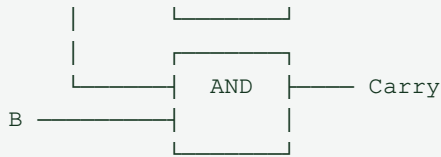
Half adder

Concept: Add two single bits. Two bits can sum to 0, 1 or 2 — and 2 doesn't fit in one bit, so you need a **Sum** output and a **Carry** output. "Half" because it cannot accept a carry *in* from a previous stage, so you can't chain them.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$\begin{aligned}\text{Sum} &= A \oplus B = A'B + AB' \\ \text{Carry} &= A \cdot B\end{aligned}$$





Gate count: 1 XOR + 1 AND (or 5 NAND gates).

Full adder

Concept: To add multi-bit numbers you need each column to accept the carry from the column to its right. A full adder has **three inputs** (A, B, Cin) and two outputs (Sum, Cout). Three input bits can total 0–3, which is exactly what 2 output bits express.

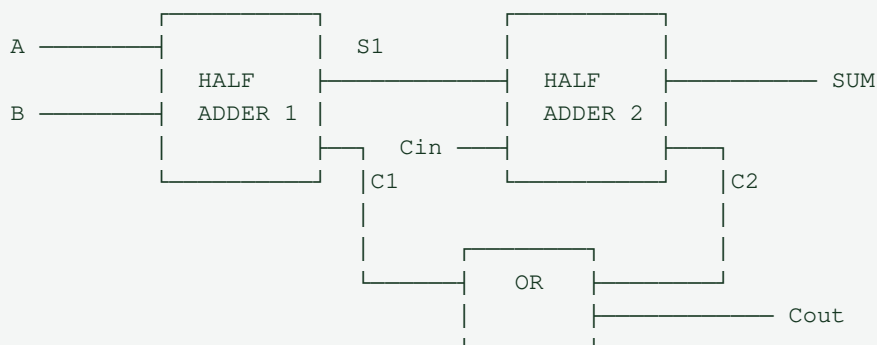
A	B	Cin	Sum	Cout	Notice
0	0	0	0	0	
0	0	1	1	0	
0	1	0	1	0	
0	1	1	0	1	
1	0	0	1	0	
1	0	1	0	1	
1	1	0	0	1	
1	1	1	1	1	Sum = 1 when an ODD number of inputs are 1

Sum = $\Sigma m(1,2,4,7)$, **Cout** = $\Sigma m(3,5,6,7)$

K-map for Sum: the 1s form a **checkerboard** — no adjacent cells — so **Sum cannot be simplified below the XOR form**. That is a nice observation to state in an exam.

$\text{Sum} = A \oplus B \oplus \text{Cin}$
 $\text{Cout} = AB + \text{BCin} + \text{ACin}$ (majority function — 1 when 2 or more inputs are 1)
 $= AB + \text{Cin}(A \oplus B)$ (the form used when building from two half adders)

Implementation using two half adders + one OR gate — draw this:

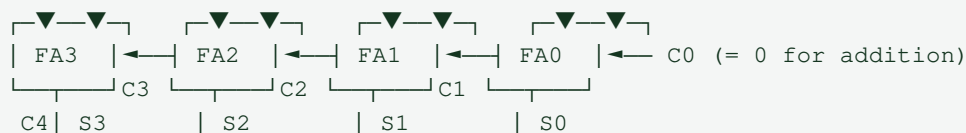


Gate count: 2 XOR + 2 AND + 1 OR (= 9 NAND gates).

4-bit ripple carry adder

Chain four full adders, carry-out of each feeding carry-in of the next.

A3	B3	A2	B2	A1	B1	A0	B0

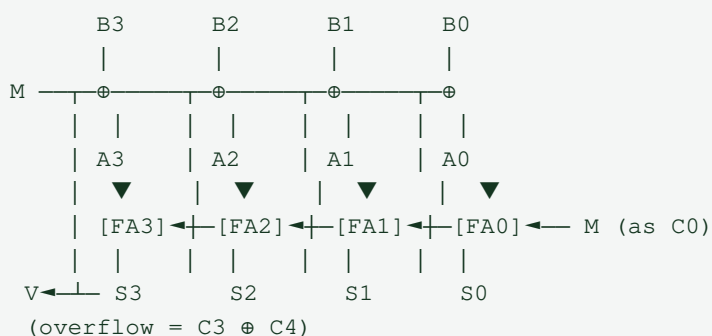


- **Problem: propagation delay.** The MSB sum is not valid until the carry has *rippled* all the way through. For n stages the worst-case delay is $\approx 2n$ gate delays (each FA contributes ~ 2 levels to the carry path). A 32-bit ripple adder is slow.
- **Solution: Carry Look-ahead Adder (CLA).** Define for each bit position:
 - **Generate** $G_i = A_i \cdot B_i$ (this stage definitely makes a carry)
 - **Propagate** $P_i = A_i \oplus B_i$ (this stage passes an incoming carry along)
 - Then $C_{i+1} = G_i + P_i \cdot C_i$, and expanding recursively gives **every** carry as a two-level function of the inputs. All carries appear simultaneously $\rightarrow$ **constant delay** (~ 3 gate delays) independent of width, at the cost of much more hardware and high fan-in.
 - IC **74283** / **7483** are 4-bit CLA adders.

Adder/Subtractor combined circuit — a classic 10-mark question

Concept: Since $A - B = A + (2\text{'s complement of } B) = A + B' + 1$, you can build one circuit that does both. Feed each B_i through an **XOR with a control line M** , and feed M into C_0 as well.

$M = 0 \rightarrow B_i \oplus 0 = B_i, \quad C_0 = 0 \rightarrow$ circuit computes $A + B$
 $M = 1 \rightarrow B_i \oplus 1 = B_i', \quad C_0 = 1 \rightarrow$ circuit computes $A + B' + 1 = A - B$



Overflow output $V = C_3 \oplus C_4$ (carry into MSB XOR carry out of MSB).

D3. Decoders

Concept

A decoder turns a **coded input into a "one-hot" output**: exactly one of its output lines goes active for each input combination. Think of it as "which one of these 8 things did you name?" — this is precisely how a memory chip selects one word out of many from an address, and how an instruction opcode selects one control line in a CPU.

An **n -to- 2^n decoder** has n inputs and 2^n outputs.

2-to-4 decoder with Enable

E	A1	A0	Y3	Y2	Y1	Y0
0	X	X	0	0	0	0
1	0	0	0	0	0	1

1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

$$\begin{aligned} Y0 &= E \cdot A1' \cdot A0' & Y1 &= E \cdot A1' \cdot A0 \\ Y2 &= E \cdot A1 \cdot A0' & Y3 &= E \cdot A1 \cdot A0 \end{aligned}$$

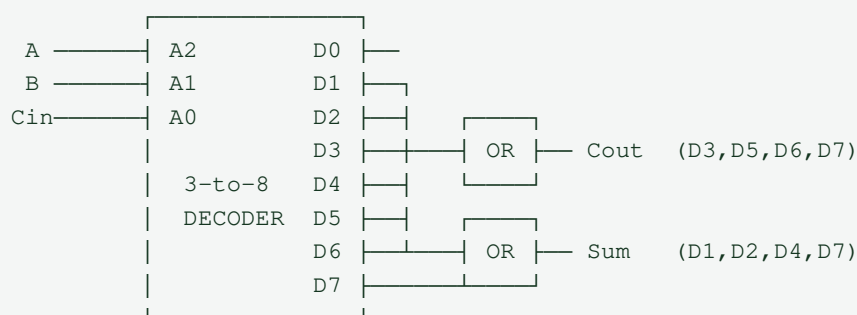
Notice: each output IS a minterm. That is the key to the next idea.

Implementing functions with a decoder

Because an n -to- 2^n decoder generates all 2^n minterms, any function of n variables can be built as an OR of the appropriate decoder outputs.

Worked example 26. Implement a full adder using a 3-to-8 decoder.

$$\begin{aligned} \text{Sum} &= \Sigma m(1, 2, 4, 7) \rightarrow \text{OR together decoder outputs } D1, D2, D4, D7 \\ \text{Cout} &= \Sigma m(3, 5, 6, 7) \rightarrow \text{OR together decoder outputs } D3, D5, D6, D7 \end{aligned}$$



If the decoder has **active-LOW** outputs (as real 74138 chips do), replace the OR gates with **NAND** gates. Very common exam refinement.

Decoder expansion

Two 3-to-8 decoders + one inverter make a **4-to-16 decoder**: use the MSB as the enable — directly to one chip, inverted to the other.

Encoders — the inverse

An encoder has **2^n inputs and n outputs**; it outputs the binary code of whichever input line is active.

8-to-3 encoder (octal-to-binary):

$$\begin{aligned} Y2 &= D4 + D5 + D6 + D7 \\ Y1 &= D2 + D3 + D6 + D7 \\ Y0 &= D1 + D3 + D5 + D7 \end{aligned}$$

(Just OR together all input lines whose index has a 1 in that bit position.)

Two problems with a plain encoder — this is what gets examined:

1. **Ambiguity if two inputs are active at once** (D3 and D6 together would produce 111 = 7, which is wrong for both).

2. **Output 000 is ambiguous**: it means either "D0 is active" or "no input is active."

Fixed with an extra **valid/V** output.

Priority encoder solves problem 1: if several inputs are active it encodes the **highest-priority (usually highest-numbered)** one and ignores the rest.

4-to-2 priority encoder truth table (D3 highest priority):

D3	D2	D1	D0	Y1	Y0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

$Y1 = D3 + D2$
 $Y0 = D3 + D2'D1$
 $V = D3 + D2 + D1 + D0$

Real-world use: interrupt priority resolution in a CPU (IC 74147 is a 10-to-4 priority encoder).

	Decoder	Encoder
Inputs → Outputs	$n \rightarrow 2^n$	$2^n \rightarrow n$
Direction	Code → one-hot	One-hot → code
Active outputs	Exactly one	The binary code
Typical use	Memory address selection, instruction decode, display drive	Keyboard scanning, interrupt priority

D4. Multiplexers and demultiplexers

Concept

A multiplexer is an **electronic rotary switch**. It has many data inputs, a few select lines, and **one output**; the select value decides which input gets connected through. It is a **data selector** — many-to-one. The classic reason to use one is to send many signals down one expensive wire (time-division multiplexing), but in digital design its *other* use is far more examinable: **a MUX can implement any Boolean function**.

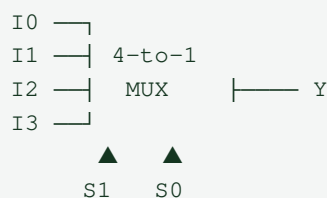
A **demultiplexer** is the same switch run backwards: one input, many outputs, select decides which output receives the data. One-to-many, a **data distributor**.

4-to-1 MUX

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

$Y = S1'S0' \cdot I0 + S1'S0 \cdot I1 + S1S0' \cdot I2 + S1S0 \cdot I3$

A 2^n -to-1 MUX needs n select lines. (8-to-1 → 3 selects; 16-to-1 → 4 selects.)



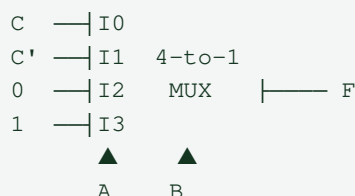
Worked example 27 — implement a 3-variable function with a 4-to-1 MUX

$$F(A,B,C) = \Sigma m(1, 2, 6, 7)$$

Method: use the two *most significant* variables (A, B) as the **select lines**. Then tabulate the function for each AB pair and express what's left in terms of C.

A	B	C	F	Select AB	Minterms	F values	Input needed
0	0	0	0	00	m0, m1	0, 1	F follows C → $I_0 = C$
0	0	1	1				
0	1	0	1	01	m2, m3	1, 0	F is opposite of C → $I_1 = C'$
0	1	1	0				
1	0	0	0	10	m4, m5	0, 0	always 0 → $I_2 = 0$
1	0	1	0				
1	1	0	1	11	m6, m7	1, 1	always 1 → $I_3 = 1$
1	1	1	1				

Answer: connect $A \rightarrow S_1$, $B \rightarrow S_0$, and $I_0=C$, $I_1=C'$, $I_2=0$, $I_3=1$.



The general rule (state it in the exam): a 2^n -to-1 MUX can implement any function of $n+1$ variables — n variables go to the select lines and the remaining variable (or its complement, or 0, or 1) goes to the data inputs. So an 8-to-1 MUX handles 4 variables, a 16-to-1 handles 5.

Demultiplexer

A 1-to-4 DEMUX:

$$Y_0 = D \cdot S_1' \cdot S_0' \quad Y_1 = D \cdot S_1' \cdot S_0 \quad Y_2 = D \cdot S_1 \cdot S_0' \quad Y_3 = D \cdot S_1 \cdot S_0$$

Key insight for MCQs: a **decoder with an enable input IS a demultiplexer** — just treat the enable pin as the data input. That is why the 74138 is documented as a "decoder/demultiplexer."

Comparison table

	MUX	DEMUX	Decoder	Encoder
--	-----	-------	---------	---------

Inputs	2^n data + n select	1 data + n select	n	2^n
Outputs	1	2^n	2^n	n
Also called	Data selector	Data distributor	Minterm generator	—
Direction	Many→one	One→many	Code→one-hot	One-hot→code

D5. Comparators, parity, code converters

Magnitude comparator

1-bit comparator:

$A > B : A \cdot B'$
 $A < B : A' \cdot B$
 $A = B : A \odot B = A'B' + AB \quad (\text{XNOR} - \text{equality is XNOR})$

n-bit comparison works from the MSB down: the numbers are equal only if **all** bit-pairs are equal (AND of all XNORs); $A > B$ if the highest-order differing bit has $A=1$.

For 4-bit:

$(A = B) = x_3 \cdot x_2 \cdot x_1 \cdot x_0 \quad \text{where } x_i = A_i \odot B_i$
 $(A > B) = A_3B_3' + x_3 \cdot A_2B_2' + x_3x_2 \cdot A_1B_1' + x_3x_2x_1 \cdot A_0B_0'$
 $(A < B) = A_3'B_3 + x_3 \cdot A_2'B_2 + x_3x_2 \cdot A_1'B_1 + x_3x_2x_1 \cdot A_0'B_0$

IC **7485** is a 4-bit magnitude comparator (cascadable).

Parity generator / checker

- **Even parity generator** for 3 data bits: $P = A \oplus B \oplus C$ (makes total number of 1s even).
- **Odd parity generator:** $P = (A \oplus B \oplus C)'$.
- **Checker:** XOR all bits including parity. Result 1 $\Rightarrow$ error (for even parity).
- Detects **any odd number** of bit errors; **cannot detect an even number** and **cannot correct** anything. IC 74180 is a 9-bit parity generator/checker.

Code converters

The general procedure: write a truth table with the **source code as inputs** and the **target code as outputs**, then K-map each output bit.

Binary → Gray (4-bit):

$G_3 = B_3$
 $G_2 = B_3 \oplus B_2$
 $G_1 = B_2 \oplus B_1$
 $G_0 = B_1 \oplus B_0$

Three XOR gates. Very cheap.

Gray → Binary (4-bit):

$B_3 = G_3$
 $B_2 = B_3 \oplus G_2 = G_3 \oplus G_2$
 $B_1 = B_2 \oplus G_1 = G_3 \oplus G_2 \oplus G_1$
 $B_0 = B_1 \oplus G_0 = G_3 \oplus G_2 \oplus G_1 \oplus G_0$

Note this is a **cascade** (each output feeds the next) so it is slower.

BCD → Excess-3 (worked truth table, with 1010–1111 as don't-cares):

Decimal	B3 B2 B1 B0	E3 E2 E1 E0
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100
10–15	—	X (don't care)

Minimising each output with a K-map (using the six don't-cares) gives the standard result:

$$\begin{aligned}
 E3 &= B3 + B2B1 + B2B0 &= B3 + B2(B1 + B0) \\
 E2 &= B2'B1 + B2'B0 + B2B1'B0' &= B2'(B1 + B0) + B2B1'B0' = B2 \oplus (B1 + B0) \\
 E1 &= B1'B0' + B1B0 &= B1 \odot B0 \quad (\text{XNOR}) \\
 E0 &= B0'
 \end{aligned}$$

Spot check, input 5 = 0101: $E3 = 0 + 1 \cdot (0+1) = 1$; $E2 = 0 \oplus (0+1) = \dots$ **careful:** $B2=1$ so $E2 = B2 \oplus (B1+B0) = 1 \oplus 1 = 0$; $E1 = B1 \odot B0 = 0 \odot 1 = 0$; $E0 = B0' = 0 \rightarrow 1000 \checkmark$ (Excess-3 for 5 is 1000).

BCD → 7-segment decoder: 4 inputs (BCD), 7 outputs (a–g). Segments are lit to form the digit. IC **7447** (common-anode, active-low outputs) / **7448** (common-cathode). Worth knowing the segment layout:



Digit 0 lights a,b,c,d,e,f (not g). Digit 1 lights only b,c. Digit 8 lights all seven.

Likely exam questions — Combinational circuits

Q	Marks
Distinguish between combinational and sequential circuits with examples.	5
Design a half adder and a half subtractor; give truth tables, expressions and logic diagrams.	5

Explain the difference between a decoder and a demultiplexer.	5
Design a full adder. Show how it can be built from two half adders and one OR gate.	10
Implement $F(A,B,C) = \Sigma m(1,2,6,7)$ using (i) a 3-to-8 decoder with OR gates and (ii) a 4-to-1 multiplexer.	10
Design a 4-bit binary adder-subtractor using full adders and XOR gates. Explain how the mode control line M selects between addition and subtraction, and how overflow is detected.	15
Explain the design procedure for combinational circuits. Using it, design a BCD-to-Excess-3 code converter, showing the truth table, K-maps for all four outputs (using don't-cares) and the final logic diagram.	15
Explain multiplexers. Show how a 2^n -to-1 MUX can realise any function of $n+1$ variables, and implement $F(A,B,C,D) = \Sigma m(0,1,3,4,8,9,15)$ using an 8-to-1 MUX.	15

MCQ traps — Combinational circuits

Trap	Truth
A 4-to-1 MUX has 4 select lines	It has 2 select lines ($2^n \rightarrow n$).
A decoder has n inputs and n outputs	n inputs, 2^n outputs.
An 8-to-1 MUX can implement 3-variable functions	It can implement 4-variable functions ($n+1$).
Half adder can be cascaded to add multi-bit numbers	It cannot — no carry input. That's the definition of "half".
Full subtractor difference $\neq$ full adder sum	They are identical : $A \oplus B \oplus C$. Only borrow/carry differ.
A ripple carry adder is fast	It is the slowest ; delay grows linearly with n. CLA fixes this.
Priority encoder and encoder are the same	Priority encoder resolves multiple simultaneous active inputs.
Decoder $\neq$ demultiplexer	A decoder with enable = demultiplexer .
Parity can correct a single-bit error	It can only detect an odd number of errors. Hamming corrects.
For n-bit binary $\rightarrow$ Gray you need n XOR gates	You need $n-1$ XORs (the MSB passes through unchanged).

PART E — SEQUENTIAL CIRCUITS